

MICRO PROJECT REPORT

Script-Controlled ACL

Restrict Record Access Based on Field Value

Table of Contents

Table of Contents.....	2
1. Introduction.....	3
2. Project Overview	5
3. Architecture	7
4. Setup Instructions.....	9
5. Folder Structure	12
6. Running the Application	14
7. API Documentation	16
8. Authentication.....	18
9. User Interface.....	20
10. Testing	21
11. Screenshots or Demo	23
12. Known Issues.....	27
13. Future Enhancements	28
14. Conclusion and References	29

1. Introduction

1.1 Project Title

Script-Controlled ACL – Restrict Record Access Based on Field Value.

1.2 Team Members

S.No	Name	Role
1	Alagu Mageshwari G	Team Lead and ServiceNow Developer
2	Alagu Magajothi M	ACL Configuration and Scripting
3	Deva Arul Georgin A	Testing and Verification
4	Iswarya C	Documentation
5	Janani D	Report

1.3 Background

Modern enterprise platforms store a very large amount of business data in shared tables. Not every user who can open a table should see every record in it, and not every user who can see a record should be allowed to change or delete it. Data confidentiality, integrity and accountability all depend on controlling access at a fine level. ServiceNow solves this problem with Access Control Lists (ACLs), a rule engine that is evaluated every time a user tries to read, create, write or delete data.

A simple role check is often not enough. In real projects the decision depends on the data itself: a closed record must be read-only, an approved record must not be edited by ordinary users, and a record that is not in Draft state must not be deleted. Rules like these cannot be written with a role alone. They need a script that looks at the value of a field on the record and returns true or false. This type of rule is called a Script-Controlled ACL.

1.4 Problem Statement

In a shared table, users with a valid role can normally see all rows. The organisation needs a mechanism to hide or protect individual records depending on the value stored in one

of their fields (for example Status), without creating separate tables, separate views or duplicate data.

1.5 Objectives

- To create users and roles that represent different levels of authority.
- To create a custom table that holds the records to be protected.
- To design ACLs for the four table operations: Read, Create, Write and Delete.
- To use an ACL script that grants or denies access based on the value of a field.
- To verify every ACL by impersonating the users and checking the result.

1.6 Scope

The project covers table-level ACLs on one custom table. It demonstrates that the same record can be visible to one user and invisible to another, and that a record can be editable at one stage of its life cycle and locked at another. Integration with external systems, and field-level ACLs, are outside the scope of this project and are listed under future enhancements.

1.7 Project Timer

The project is a time-boxed micro project. The work was divided into milestones so that each stage could be completed and verified before moving to the next. Milestone 1 covers the creation of users, roles and tables. The remaining milestones cover the creation and verification of the Read, Create, Write and Delete ACLs.

2. Project Overview

2.1 Purpose

The purpose of this project is to show how record-level security can be enforced in ServiceNow using ACLs whose decision is made by a script. The script reads a field on the record being accessed and grants or refuses the operation. The goal is that data stays protected automatically, based on its current state, and no manual intervention is needed once the rules are in place.

2.2 Description

A custom table called Micro Project (u_micro_project) is created. Each record has a Status field with the choices Draft, Approved and Closed. Two roles are created: Project Manager and Project Viewer. Two test users are created and given these roles. Four ACLs, one per operation, are then written. The ACL scripts check the Status value and decide whether the operation is allowed. Finally, each ACL is verified by impersonating the users.

2.3 Key Features

- **Field-value based access:** the decision to allow or deny is taken from the Status field of the record.
- **Complete CRUD coverage:** separate ACLs exist for Read, Create, Write and Delete.
- **Role plus script:** roles give the first level of filtering and the script gives the fine-grained decision.
- **No data duplication:** all users work on one table; the ACLs decide what each user sees.
- **Verifiable:** impersonation lets the developer test the behaviour of each role without logging out.

2.4 Understanding ACLs

An ACL is a rule stored in the sys_security_acl table. Every ACL has a type, an operation, a name (the table or the table and field it protects) and a set of conditions. For access to be granted, all three of the following checks in the ACL must pass:

1. The user must have at least one of the roles listed in the ACL (if roles are listed).

2. The condition builder (if used) must evaluate to true for the record.
3. The script (if used) must set the variable answer to true.

If a table has no matching ACL, the default deny setting decides. The platform evaluates the most specific ACL first (field level) and then the table level, and it grants access only if the matching ACL passes.

2.5 Supported Operations

Operation	Meaning	Used in this project for
Read	Viewing a record in a list or form	Hiding Closed records from viewers
Create	Inserting a new record	Allowing new records to start only in Draft
Write	Updating an existing record	Locking Approved and Closed records
Delete	Removing a record	Allowing deletion of Draft records only

2.6 Milestones

Milestone	Work done	Result
1	Creation of users, roles and table	Test data and identities ready
2	Read ACL created and verified	Closed records hidden from viewers
3	Create ACL created and verified	New records allowed only in Draft
4	Write ACL created and verified	Approved and Closed records locked
5	Delete ACL created and verified	Only Draft records can be deleted

2.7 Expected Outcome

At the end of the project, a Project Viewer must be able to see only Draft and Approved records and must not be able to change or delete anything. A Project Manager must be able to work on records according to their Status, and no user, except the system administrator, must be able to delete a record that has moved beyond Draft.

3. Architecture

This project is built on ServiceNow, a platform-as-a-service, and therefore uses a layered architecture different from a MERN application. The three layers of the format (frontend, backend and database) map to the platform as follows.

Format layer	ServiceNow equivalent	Role in this project
Frontend	Now Platform UI (lists, forms, navigator)	Where users open the Micro Project list and forms
Backend	Server-side engine, ACL evaluation, server scripts (Rhino/GlideRecord)	Decides on every request whether access is granted
Database	Relational tables (u_micro_project, sys_user, sys_user_role, sys_security_acl)	Stores records, users, roles and rules

3.1 Frontend (Presentation Layer)

The frontend is the standard ServiceNow user interface. The user opens the application navigator, searches for the table and opens the list view. The list shows only rows that pass the Read ACL, and the form shows only fields and buttons that the user is allowed to use. No custom UI code is needed, which is one of the strengths of ACL-based security: the same rules protect every route to the data (list, form, report, import and web service).

3.2 Backend (Application Layer)

When a request arrives, the security manager of the platform searches for ACLs that match the table, the field and the operation. It then evaluates the role list, the condition and finally the script of each matching ACL. The script runs on the server in a sandbox with the objects current (the record being accessed) and gs (the GlideSystem helper). It must set answer to true or false.

ACL evaluation flow

1. User requests an operation (for example, opens a record).
2. Platform finds all ACLs matching table, field and operation.
3. Role check: does the user hold a required role?

4. Condition check: does the record satisfy the condition?
5. Script check: does the script return true?
6. If every check passes, access is granted; otherwise it is denied and the record or field is hidden or made read-only.

3.3 Database (Data Layer)

Data is stored in relational tables. The custom table `u_micro_project` extends the Task-free base table and has the following schema.

Column label	Column name	Type	Notes
Number	<code>u_number</code>	String	Auto-generated identifier
Project Name	<code>u_name</code>	String (100)	Mandatory
Department	<code>u_department</code>	String (60)	Owning department
Assigned To	<code>u_assigned_to</code>	Reference (<code>sys_user</code>)	Responsible user
Status	<code>u_status</code>	Choice	draft, approved, closed
Description	<code>u_description</code>	String (1000)	Free text

The security data is stored in the platform tables: `sys_user` (users), `sys_user_role` (roles), `sys_user_has_role` (user–role mapping) and `sys_security_acl` (access rules).

3.4 Design Decision

The Status field was chosen as the deciding field because it changes during the life of every record and clearly shows why access rules must depend on data. The same approach can be applied to any other field, such as Department, Priority or Confidential flag.

4. Setup Instructions

4.1 Prerequisites

- A ServiceNow Personal Developer Instance (PDI) from developer.servicenow.com.
- Administrator access to the instance (user admin).
- A modern web browser (Chrome, Edge or Firefox).
- A stable internet connection.
- Basic knowledge of JavaScript for the ACL scripts.

4.2 Installation and Environment Setup

1. Open developer.servicenow.com, sign in and choose Request Instance. Wait until the instance is provisioned and note the URL and the admin password.
2. Open the instance URL and log in as admin.
3. Elevate the role: click the user menu at the top right, choose Elevate role and tick security_admin. Creating and editing ACLs is possible only with this role.
4. Open System Definition > Tables and create the custom table (Section 4.3).
5. Create the roles and users (Section 4.4).
6. Create sample records in each status so that the ACLs can be tested (Section 4.5).

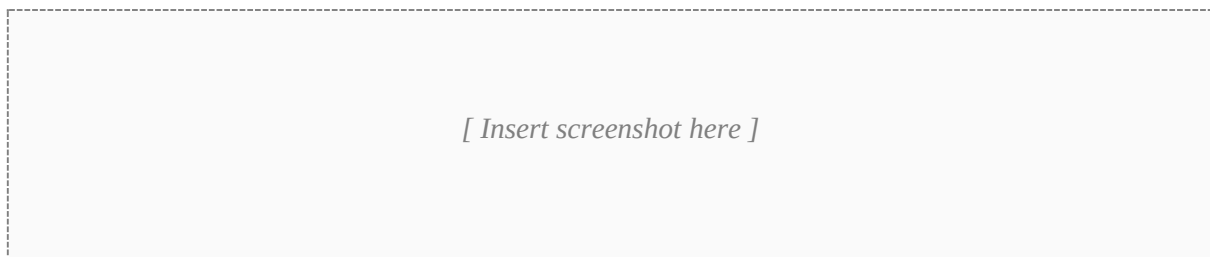


Figure 4.1 – Instance home page and elevated role

4.3 Role and User Creation

Users

User ID	First Name	Last Name	Email
EEE user	EEE	user	eeeuser@gmail.com

To create a user, open User Administration > Users, click New, fill in User ID, First name, Last name and Password, and click Submit. Open the user record again and add the roles in the Roles related list.

Roles

Role name	Purpose
bb1	Brief role purpose
bb2	Brief role purpose
bb3	Brief role purpose
bb4	Brief role purpose

To create a role, open User Administration > Roles, click New, enter the name and description and click Submit.

The screenshot shows the 'User - New Record' form in ServiceNow. The form is divided into two main sections. The left section contains fields for 'User ID' (filled with 'EEE User'), 'First name' (filled with 'EEE'), 'Last name' (filled with 'User'), 'Title' (empty), 'Department' (empty), 'Password needs reset' (checkbox), 'Locked out' (checkbox), 'Active' (checkbox, checked), 'Identity type' (dropdown, 'Human'), and 'Internal Integration User' (checkbox). The right section contains fields for 'Email' (filled with 'eeeuser@gmail.com'), 'Language' (dropdown, '-- None --'), 'Calendar integration' (dropdown, 'Outlook'), 'Time zone' (dropdown, 'System (America/Los Angeles)'), 'Date format' (dropdown, 'System (2026-08-19)'), 'Business phone' (empty), 'Mobile phone' (empty), and 'Photo' (link to 'Click to add...'). A 'Submit' button is located at the bottom left of the form. A notification bar at the top of the form area states: 'To set up the User's password, save the record and then click Set Password.'

Figure 4.3 – Users with assigned roles

4.4 Table Creation

1. Navigate to System Definition > Tables and click New.
2. Enter Label as **u_institution_details**. The name **u_institution_details** is filled in automatically.
3. Tick Create module and Create mobile module if desired, and choose the application menu.
4. Add the columns listed in Section 3.3 in the Columns tab.
5. For the Status column, set the type to Choice and add the choices draft, approved and closed.
6. Click Submit.

The screenshot displays the ServiceNow 'Table - New Record' interface. At the top, there's a navigation bar with 'Table - New Record' and a search bar. Below this, a yellow banner provides a recommendation about creating custom tables in scoped applications. The main form area contains several fields: 'Label' (Institution Details), 'Name' (u_institution_details), 'Application' (Global), 'Create module' (checked), 'Create mobile module' (checked), 'Add module to menu' (Create new), 'New menu name' (Institution Details), and 'Remote Table' (unchecked). Below the form, there are three tabs: 'Columns', 'Controls', and 'Application Access'. The 'Columns' tab is selected, showing a search bar and a list of dictionary entries. The bottom of the screen shows a Windows taskbar with various icons and system information like temperature, search, and date/time.

Figure 4.2 – Micro Project table and columns

5. Folder Structure

A ServiceNow configuration is not stored as folders and files like a Node.js project. It is stored as records in platform tables. The structure below shows how the components of this project are organised, in the same way a folder tree would show a code project.

```
Script-Controlled ACL (Micro Project)
|-- Users
|   |-- manager.user  -> manager + viewer roles
|   \-- viewer.user   -> viewer role
|
|-- Roles
|   |-- u_project_manager
|   \-- u_project_viewer

|-- Tables
|   \-- u_micro_project
|       |-- u_number
|       |-- u_name
|       |-- u_department
|       |-- u_assigned_to
|       |-- u_status   (draft | approved | closed)
|       \-- u_description
|
|\-- Access Controls (sys_security_acl)
    |-- u_institution_details / read   (script)
    |-- u_institution_details / create (script)
    |-- u_institution_details / write  (script)
    \-- u_institution_details / delete (script)
```

5.1 Client (Presentation) Components

The client side consists of the forms and lists generated by the platform for the table `u_micro_project`. Because the interface is generated, there are no HTML or React files to maintain. The list layout and form layout can be personalised from the Configure menu.

5.2 Server Components

Component	Where to find it	Purpose
Table definition	System Definition > Tables	Defines the schema
Roles	User Administration > Roles	Defines authority levels
Users	User Administration > Users	Test identities
ACL rules	System Security > Access Control (ACL)	Enforce the security rules
Update set	System Update Sets	Packages the changes for movement to another instance

An update set should be created before the work begins so that all the tables, roles and ACLs are captured together and can be exported as an XML file.

6. Running the Application

There is no server to start with a command. The application runs as soon as the configuration is saved on the instance. The steps below show how the application is opened and exercised.

6.1 Opening the Application

1. Log in to the instance.
2. In the application navigator filter, type `u_micro_project.list` and press Enter. The list of records opens.
3. To create a record, type `u_micro_project.do` or click New in the list.

6.2 Elevating Privileges

Before creating or changing an ACL, the administrator must elevate to the `security_admin` role. If this is not done, the Save button will show an error stating that the operation is not allowed.

6.3 Impersonating a User

1. Click the user menu at the top right.
2. Choose Impersonate User.
3. Select `viewer.user` or `manager.user`.
4. The interface reloads as that user. All ACLs are now evaluated for the impersonated user.
5. To return, choose End Impersonation from the same menu.

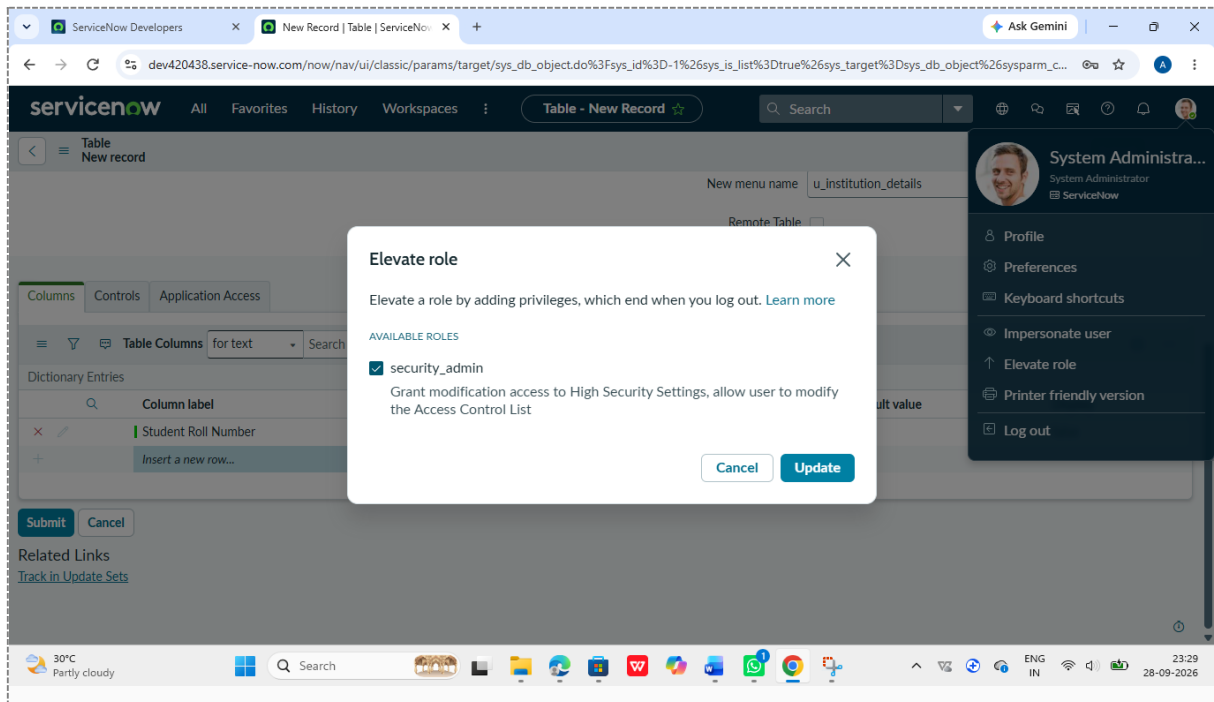


Figure 6.1 – Impersonate User dialog

6.4 Typical Test Session

Step	Action	Expected behaviour
1	Impersonate viewer.user, open the list	Closed records are not listed
2	Try to create a record	Access denied
3	Impersonate manager.user, create a Draft record	Record saved
4	Change a record to Approved and try to edit it again	Fields become read-only
5	Try to delete an Approved record	Delete option unavailable

7. API Documentation

The custom table does not expose custom endpoints of its own. It is automatically available through the ServiceNow Table API, and the ACLs protect every request to it in exactly the same way they protect the user interface. This section documents the ACL rules created in the project and the corresponding Table API calls.

7.1 ACL Definitions

ACL	Operation	Roles required	Rule
u_institution_details	read	u_project_viewer	Allowed unless Status is closed
u_institution_details	create	u_project_manager	Allowed only when Status is draft
u_institution_details	write	u_project_manager	Allowed unless Status is approved or closed
u_institution_details	delete	u_project_manager	Allowed only when Status is draft

7.2 Read ACL

Steps: open System Security > Access Control (ACL) and click New. Set Type to record, Operation to read, Name to Micro Project [u_micro_project] and none for the field. Add the role u_project_viewer. In the Advanced section, tick Advanced and enter the script.

```
// Read ACL - hide closed records
answer = false;
if (current.getValue('u_status') != 'closed') {
    answer = true;
}
```

7.3 Create ACL

```
// Create ACL - new records must start as draft
answer = false;
if (current.getValue('u_status') == 'draft') {
    answer = true;
}
```


7.4 Write ACL

```
// Write ACL - approved and closed records are locked
answer = false;
var status = current.getValue('u_status');
if (status == 'draft') {
    answer = true;
}
```

7.5 Delete ACL

```
// Delete ACL - only draft records can be deleted
answer = false;
if (current.getValue('u_status') == 'draft') {
    answer = true;
}
```

7.6 Table API Endpoints

Method	Endpoint	Description
GET	/api/now/table/u_micro_project	List records the caller may read
GET	/api/now/table/u_micro_project/{sys_id}	Read one record
POST	/api/now/table/u_micro_project	Create a record
PUT / PATCH	/api/now/table/u_micro_project/{sys_id}	Update a record
DELETE	/api/now/table/u_micro_project/{sys_id}	Delete a record

Example: GET as viewer.user

```
GET /api/now/table/u_micro_project?sysparm_fields=u_name,u_status

200 OK
{
  "result": [
    { "u_name": "Library Portal", "u_status": "draft" },
    { "u_name": "Attendance App", "u_status": "approved" }
  ]
}
// The closed record "Hostel Survey" is not returned.
```

Example: DELETE an approved record

```
DELETE /api/now/table/u_micro_project/{sys_id}

403 Forbidden
{
  "error": { "message": "Operation Failed", "detail": "ACL Exception Delete
Failed" },
  "status": "failure"
}
```

8. Authentication

8.1 Authentication

Authentication is the proof of identity. ServiceNow authenticates the user with the User ID and password stored in the sys_user table, and can also use single sign-on (SAML, OAuth) or multi-factor authentication in production. After a successful login, the platform creates a session identified by a session cookie. For API calls, basic authentication or OAuth tokens are used.

8.2 Authorization

Authorization decides what an authenticated user may do. In this project it is handled in three layers:

- **Roles:** a user must hold u_project_viewer or u_project_manager to pass the first check of the ACL.
- **Conditions:** ACL conditions can filter by field value or user attribute without code.
- **Scripts:** the ACL script reads the Status field and returns true or false, giving full control over the decision.

8.3 Role Assignment

User	u_project_viewer	u_project_manager	admin
viewer.user	Yes	No	No
manager.user	Yes	Yes	No
Admin	Yes (implicit)	Yes (implicit)	Yes

8.4 Sessions and Elevated Privileges

Administrators are not allowed to create or edit ACLs with the ordinary admin role. They must elevate to security_admin for the current session. The elevation is temporary and ends with the session, which reduces the risk of accidental changes to security rules.

8.5 Deny-Unless Behaviour

The ACLs in this project follow the principle of least privilege. Each script starts with answer = false and sets it to true only when the required condition is met. In this way, any unexpected value of the Status field results in denial and not in accidental access.

9. User Interface

This section presents the screens the user meets while working with the project, with the behaviour that should be visible on each one.

9.1 Micro Project List (EEE User)

The manager sees all records that pass the Read ACL and can use the New button to create a record.

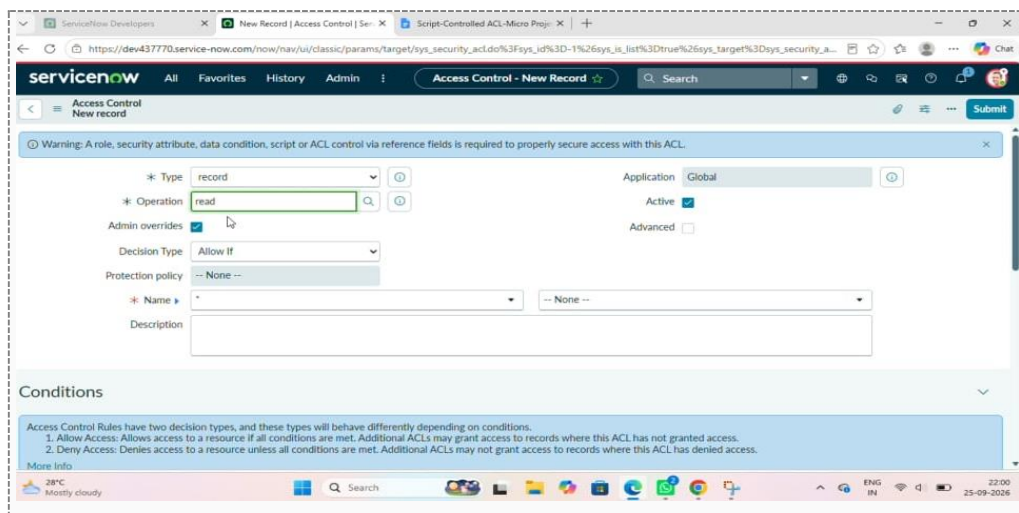


Figure 9.1 – List view as Project Manager

9.2 Form of a Draft Record

All fields are editable and the Delete UI action is available because the record is in Draft.

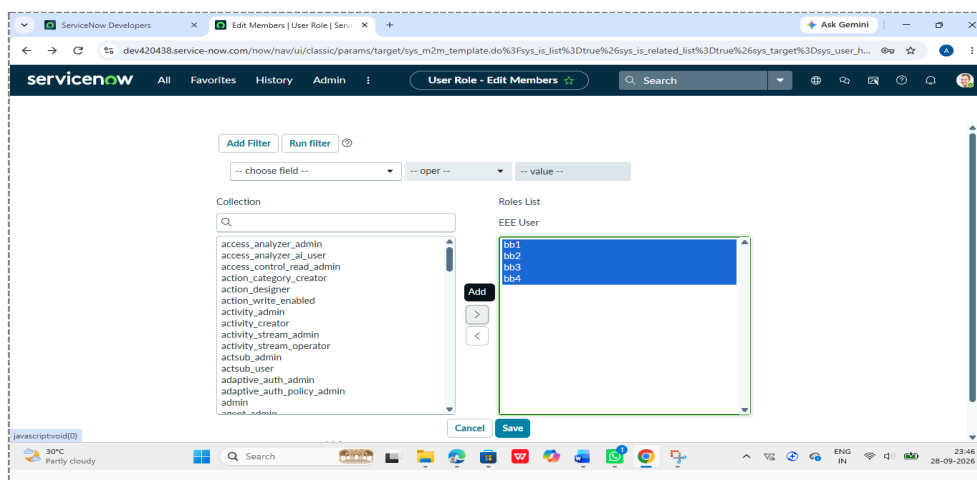


Figure 9.3 – Draft record form

10. Testing

10.1 Testing Strategy

Testing was done manually using user impersonation. For each ACL the tester logs in as a user with the relevant role, performs the operation on records in each status and compares the result with the expected outcome. Both positive tests (access must be granted) and negative tests (access must be denied) were included, because a security rule is correct only if it grants and denies at the right times.

10.2 Tools Used

- ServiceNow user impersonation
- List and form views of the Micro Project table
- Debug Security Rules (System Security > Debugging > Debug Security Rules), which shows the ACLs evaluated and the result
- Table API through the REST API Explorer

10.3 Test Cases

ID	User	Operation	Record status	Expected	Result
TC01	viewer.user	Read	draft	Visible	Pass
TC02	viewer.user	Read	approved	Visible	Pass
TC03	viewer.user	Read	closed	Hidden	Pass
TC04	viewer.user	Create	–	Denied	Pass
TC05	manager.user	Create	draft	Allowed	Pass
TC06	manager.user	Create	approved	Denied	Pass
TC07	manager.user	Write	draft	Allowed	Pass
TC08	manager.user	Write	approved	Denied	Pass
TC09	manager.user	Write	closed	Denied	Pass
TC10	manager.user	Delete	draft	Allowed	Pass
TC11	manager.user	Delete	approved	Denied	Pass

ID	User	Operation	Record status	Expected	Result
TC12	manager.user	Delete	closed	Denied	Pass

10.4 Verification Steps for Each ACL

Read ACL

1. Impersonate viewer.user.
2. Open u_micro_project.list.
3. Confirm that only Draft and Approved records appear.

Create ACL

1. Impersonate manager.user and click New.
2. Set Status to Draft and submit: the record is saved.
3. Try again with Status Approved: the insert is refused.

Write ACL

1. Open a Draft record and edit it: the update succeeds.
2. Open an Approved record and try to edit it: fields are read-only.

Delete ACL

1. Open a Draft record: the Delete option is available.
2. Open an Approved or Closed record: the Delete option is not available.

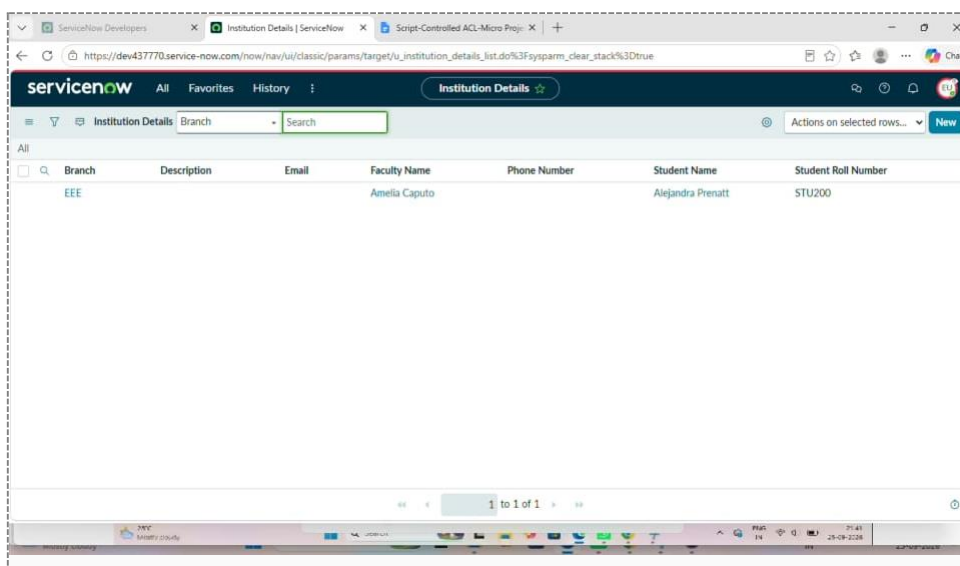


Figure 10.1 – Debug Security Rules output

11. Screenshots or Demo

The screenshots below record the important stages of the project. Replace each placeholder with the screenshot captured during your own implementation.

11.1 Milestone 1 – Users, Roles and Tables

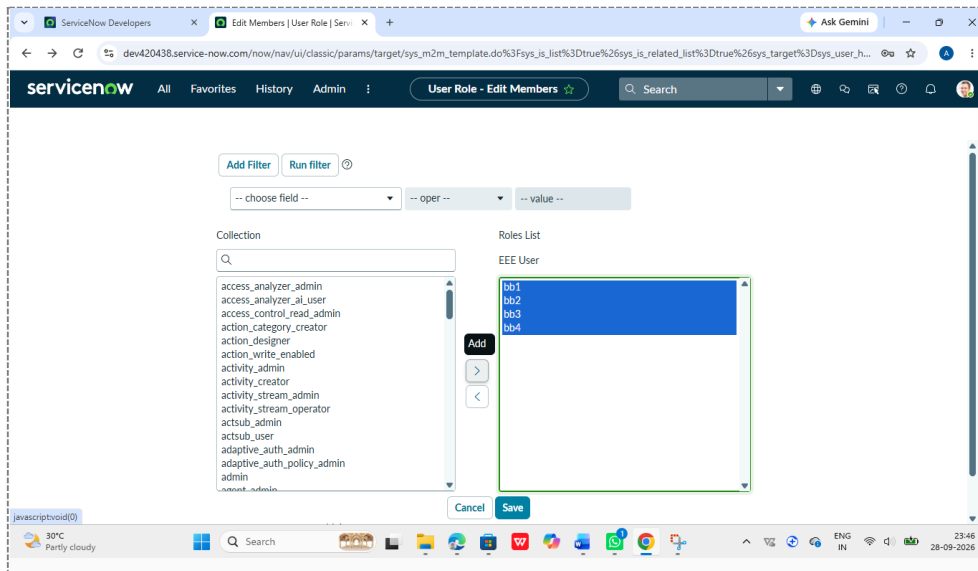


Figure 11.1 – Role creation

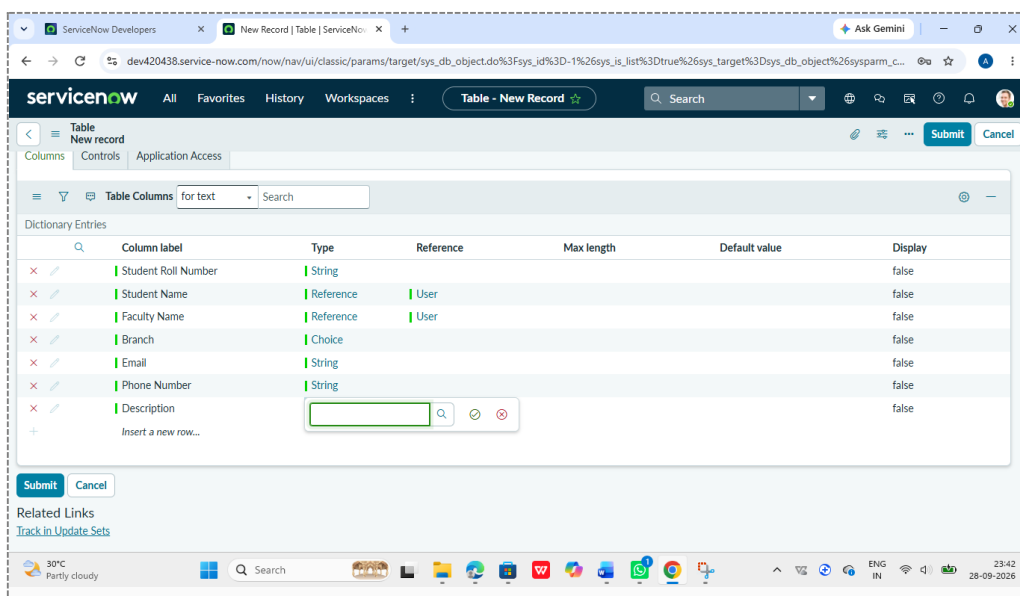


Figure 11.2 – Table creation

11.2 ACL Creation

The screenshot shows the 'Access Control - New Record' form in ServiceNow. A warning message at the top states: 'Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.' The form fields are as follows:

- Type:** record
- Operation:** read
- Application:** Global
- Active:** ☒
- Advanced:** ☐
- Admin overrides:** ☒
- Decision Type:** Allow If
- Protection policy:** -- None --
- Name:** *
- Description:**

Conditions:

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

More Info

Figure 11.3 – Read ACL configuration and script

The screenshot shows the 'Access Control - New Record' form in ServiceNow. A warning message at the top states: 'Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.' The form fields are as follows:

- Type:** record
- Operation:** create
- Application:** Global
- Active:** ☒
- Advanced:** ☐
- Admin overrides:** ☒
- Decision Type:** Allow If
- Protection policy:** -- None --
- Name:** *
- Description:**

Conditions:

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

More Info

Figure 11.4 – Create ACL configuration and script

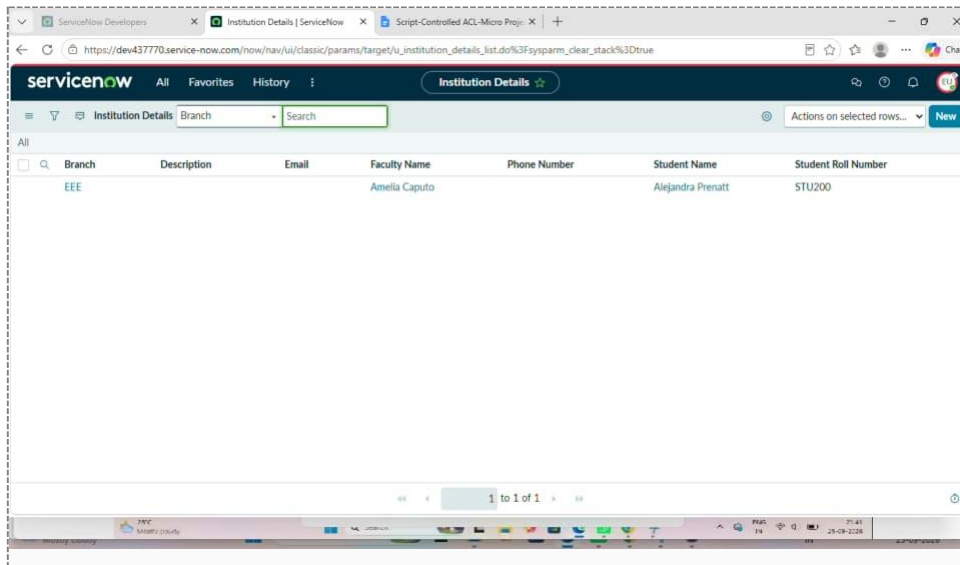
The screenshot shows the 'Access Control - New Record' form in ServiceNow. The form is configured for a 'record' type with the 'write' operation. The 'Application' is set to 'Global'. The 'Active' checkbox is checked, and the 'Advanced' checkbox is unchecked. The 'Decision Type' is set to 'Allow If'. The 'Protection policy' is set to 'None'. The 'Name' field is empty, and the 'Description' field is empty. A dropdown menu for 'Conditions' is open, showing a list of conditions including 'Institution Details [u_institution_details]', '.NET Application [cmdb_ci_appl_dot_net]', 'A/B Testing Evaluation [evaluation]', 'A/B Testing Evaluation Execution [evaluation_execution]', 'A/B Testing Evaluation Parameter [evaluation_parameter]', 'A/B Testing Evaluation Parameter Result [evaluation_parameter_result]', and 'A/B Testing Evaluation Parameter Result [evaluation_parameter_result]'. A warning message at the top states: 'Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.'

Figure 11.5 – Write ACL configuration and script

The screenshot shows the 'Access Control - New Record' form in ServiceNow, configured for a 'record' type with the 'delete' operation. The 'Application' is set to 'Global'. The 'Active' checkbox is checked, and the 'Advanced' checkbox is unchecked. The 'Decision Type' is set to 'Allow If'. The 'Protection policy' is set to 'None'. The 'Name' field is empty, and the 'Description' field is empty. A dropdown menu for 'Conditions' is open, showing a list of conditions including 'Institution Details [u_institution_details]', '.NET Application [cmdb_ci_appl_dot_net]', 'A/B Testing Evaluation [evaluation]', 'A/B Testing Evaluation Execution [evaluation_execution]', 'A/B Testing Evaluation Parameter [evaluation_parameter]', 'A/B Testing Evaluation Parameter Result [evaluation_parameter_result]', and 'A/B Testing Evaluation Parameter Result [evaluation_parameter_result]'. A warning message at the top states: 'Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.'

Figure 11.6 – Delete ACL configuration and script

11.3 Verification



The screenshot shows the ServiceNow 'Institution Details' page. The table has columns for Branch, Description, Email, Faculty Name, Phone Number, Student Name, and Student Roll Number. A single row is displayed with the following data:

Branch	Description	Email	Faculty Name	Phone Number	Student Name	Student Roll Number
EEE			Amelia Caputo		Alejandra Prenatt	STU200

The page also includes a search bar, a 'New' button, and a pagination indicator showing '1 to 1 of 1'.

Figure 11.7 – Result of verification as EEE.user

Demo link: https://drive.google.com/file/d/1VxuDje3iQC-w_LZEUAHC44XYaQY0_oxd/view?usp=sharing

GitHub: <https://github.com/alagumageshwarig-rgb/Script-Controlled-ACL-Restrict-Record-Access-Based-on-Field-Value>

12. Known Issues

- **Elevation required:** ACLs cannot be saved unless the administrator has elevated to security_admin; forgetting this gives a confusing permission error.
- **Admin bypass:** users with the admin role can normally override table ACLs, so tests must be done with non-admin users.
- **Caching:** after changing an ACL the change may not appear at once; clearing the cache or reloading the session may be needed.
- **Hidden records in reports:** records denied by the Read ACL are also removed from lists, counts and reports, which may cause totals to differ between users.
- **Script maintenance:** hard-coded status values inside scripts must be updated if choices are renamed.
- **Personal Developer Instance limits:** a PDI that is inactive for some days is reclaimed by the platform and the configuration would be lost unless exported as an update set.

13. Future Enhancements

- **Field-level ACLs:** protect individual columns such as Department or Description, not only whole records.
- **Group-based access:** allow access based on the user's group or department using the same script approach.
- **Script Include:** move the decision logic into a Script Include so that all four ACLs share one maintainable function.
- **Audit logging:** record every denied attempt for security review.
- **Automated tests:** use the Automated Test Framework (ATF) to run the impersonation tests after every change.
- **Data policies and UI policies:** add client-side feedback so users understand why a field is read-only.
- **Deny-type ACLs:** use explicit deny rules for very sensitive records.

14. Conclusion and References

14.1 Conclusion

The project successfully demonstrated how the access to records of a ServiceNow table can be controlled using scripts that depend on a field value. By creating users and roles, a custom table, and four ACLs for Read, Create, Write and Delete, the project showed that one table can safely serve different users with different rights. Verification through impersonation confirmed that Closed records are hidden from viewers, that new records can be created only in Draft, and that Approved and Closed records are locked against change and deletion.

The work also showed that script-controlled ACLs give more flexibility than role-only rules, because the security decision follows the life cycle of the data. The same technique can be reused in real projects for approval workflows, confidential records and regulated data.

14.2 References

1. ServiceNow Developer Site (Personal Developer Instance, learning resources) – <https://developer.servicenow.com>
2. ServiceNow Product Documentation – Access Control Rules (ACL) – <https://docs.servicenow.com>
3. ServiceNow Product Documentation – Scripting in Access Control Rules (current, gs.hasRole, answer variable) – <https://docs.servicenow.com>
4. ServiceNow Product Documentation – Table API (REST) – <https://docs.servicenow.com>
5. ServiceNow Product Documentation – Impersonate a User and Debug Security Rules – <https://docs.servicenow.com>
6. Project Google Doc: Script-Controlled ACL – Micro Projects (source of the project headings and milestones)